

Memory Controllers for DDR Memories

Mentor: Mahesh A

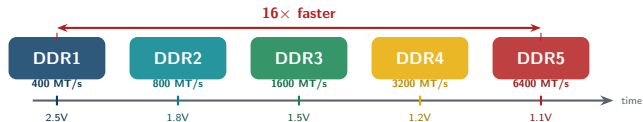
Pranav M PES2UG23EC076

Lalith PES2UG23EC074

Harshini S PES2UG23EC058

Recap: Memory Controller for DDR Memories

- Acts as a bridge between the SoC and DRAM
- Enforces 50+ timing constraints (tWTR, tRC, tFAW...)
- Handles row-buffer management and request scheduling
- Manages power modes: PASR, TCSR, Deep Power-Down
- AXI out-of-order IDs map naturally to DDR bank parallelism



	DDR1	DDR2	DDR3	DDR4	DDR5
Prefetch	2n	4n	8n	8n	16n
Banks	4	4	8	16	32

DDR3 / DDR4 / DDR5 — JEDEC Key Specs

Parameter	DDR3 (JESD79-3F)	DDR4 (JESD79-4C)	DDR5 (JESD79-5B)
Data Rate (MT/s)	800–2133	1600–3200	3200–6400+
Prefetch / BL	8n / BL8 (fixed/fly)	8n / BL8 (fixed/fly)	16n / BL16 mandatory
Bank Groups	None	2 or 4 BGs	8 BGs (sub-ch.)
Channels	Single	Single	Sub-channel ×2
IO Voltage	1.5 V / 1.35 V	1.2 V	1.1 V
On-Die ECC	Optional	Optional	Required (×4 org)
Refresh	tREFI = 7.8 μs	tREFI = 7.8 μs	Dual-rate / PBR
Termination	RTT_NOM	+ RTT_PARK	+ DFITMG
New timing	tRCD, tRP, tRAS, tWR, tRFC	+ tWTR_L/S, tRRD_L/S	+ tRFC1/2/sb, tCS

DDR3 JEDEC Deep-Dive

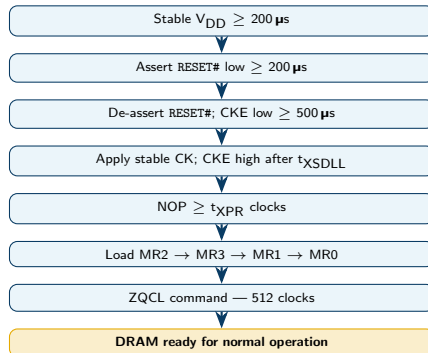
Critical Timing Parameters

tRCD	Row open → column command
tRP	Precharge → next ACT
tRAS	Min row active time
tRC	tRAS + tRP (full row cycle)
tWR	Write → precharge recovery
tRFC	Refresh cycle (density-dependent)
tFAW	4-bank activation window
tWTR	Write → read turnaround
CL	CAS latency (cmd → first data)

Mode Registers (MRS)

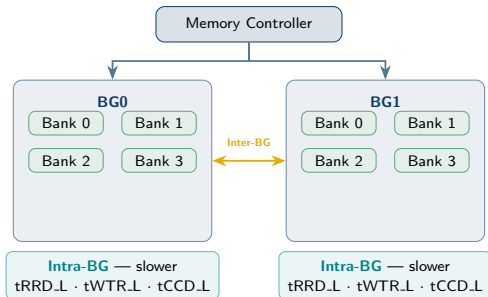
MR0	CL, BL, DLL reset, write recovery
MR1	Drive strength, RTT_NOM, AL
MR2	CWL, ASR, SRT, RTT_WR
MR3	MPR (multi-purpose register)

Initialization Sequence



DDR4 JEDEC Deep-Dive

Bank Group Architecture

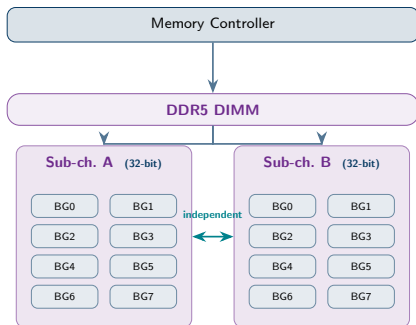


PHY Training Procedures

- 1 **Write Levelling:** DQS launched; DRAM echoes on rising CK; adjusts DQS-to-CK skew per lane
- 2 **Read Gate:** PHY determines preamble window; gates DQS capture
- 3 **Read Centering:** Sweeps DQ VREF; finds eye centre
- 4 **Write Centering:** Tunes PHY DQ delay vs DQS
- 5 **ZQ Cal:** Periodic ZQCS; updates R_{ZQ} impedance code

DDR5 JEDEC Deep-Dive

Sub-Channel Architecture

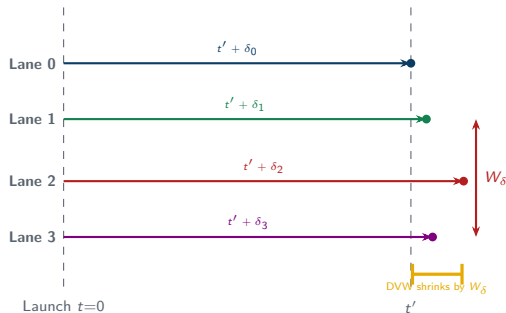


Init & Training Additions

- 1 **BCOM training:** Sideband cmd bus trained before main CA bus active
- 2 **PDA mode:** Per-DRAM addressability — tune MRS per individual device
- 3 **2× MRS sets:** Extended register space; extra latency + ODT configs
- 4 **RTT_PARK + DFITMG:** Runtime-switchable ODT via DFI timing params
- 5 **CA parity:** DRAM validates each command frame; error flagged via ALERT_n

Inter-Lane Skew: Off-Die Signal Integrity

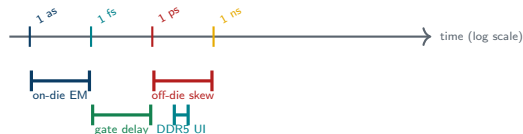
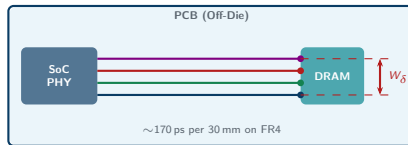
- DDR transfers data in parallel across multiple DQ byte lanes
- Each lane follows a different PCB path, picking up delay δ_i
- Skew window:
 $W_\delta = \max(\delta_i) - \min(\delta_i)$
- DDR5 UI ≈ 156 ps; 20 ps skew eats >12% of it
- PHY training (Write Levelling, DQS Gating) corrects δ_i per lane



Off-Die vs On-Die Delay

Off-Die (Picoseconds)

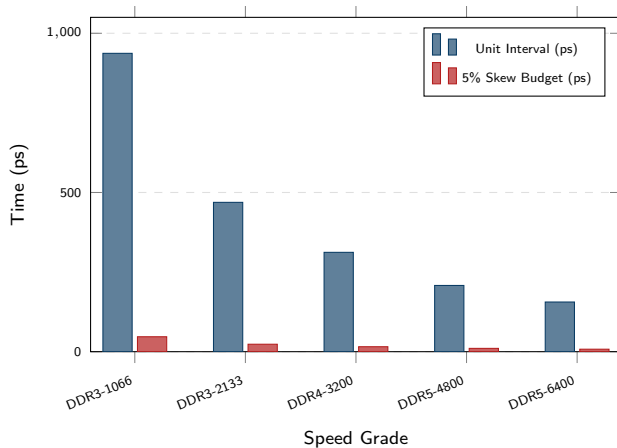
- Signals traverse PCB traces, vias
- FR4 velocity $\approx 0.6c$; ~ 1 ps per $180 \mu\text{m}$
- 30 mm trace ≈ 170 ps; W_δ typically 5–50 ps



On-Die (Femto/Attoseconds)

- nm-scale on-chip metal interconnect
- $1 \mu\text{m}$ wire $\approx 3\text{--}10$ as propagation
- Gate delay (3 nm): 1–5 ps; bottleneck is logic depth

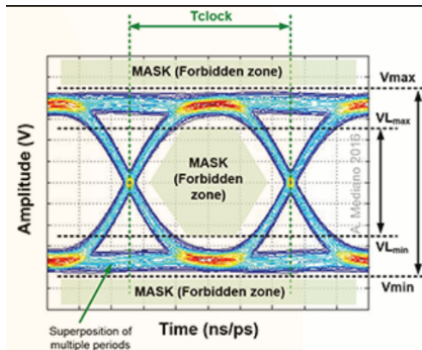
Why Picoseconds Matter at Modern DDR Speeds



Speed	UI	5% Budget
DDR3-1066	937 ps	46.9 ps
DDR3-2133	469 ps	23.4 ps
DDR4-3200	312 ps	15.6 ps
DDR5-4800	208 ps	10.4 ps
DDR5-6400	156 ps	7.8 ps

- At DDR5-6400 skew budget is just **7.8 ps**
- Uncompensated off-die skew can fill this entirely
- PHY training trims δ_i to $\ll 5$ ps per lane

Eye Diagram Fundamentals



Key Regions

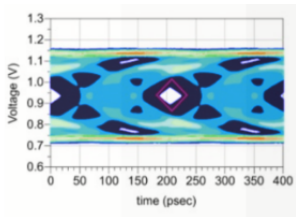
- **Eye Width:** horizontal opening between transition windows
- **Transition Window:** region where 0→1 or 1→0 switching occurs
- **Steady-State Window:** stable HIGH or LOW; sampled here
- **Unit Interval (UI):** one bit period; eye must open within it

Why it matters

- Receiver samples at eye centre for maximum margin
- Noise and jitter reduce eye opening
- At DDR5-6400: UI = 156 ps — any skew directly closes the eye

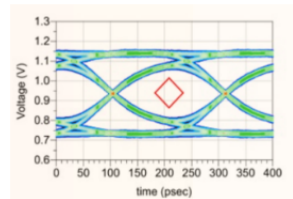
Eye Diagram: Trained vs Untrained

Untrained Eye



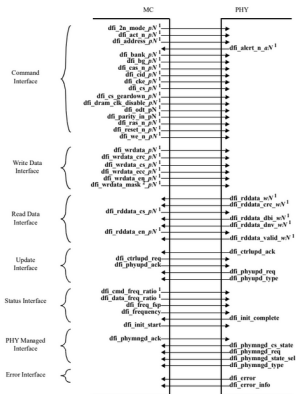
- Lane skew δ_i closes eye horizontally
- VREF offset closes eye vertically
- ISI and jitter blur transition edges
- High BER — unreliable data capture

Trained & Calibrated Eye



- Write Levelling corrects DQS-to-CK skew δ_i
- Read centering places sample point at eye centre
- ZQ calibration sets correct driver impedance
- 2D VREF scan optimises voltage margin M_V

DFI Interface: MC ↔ PHY Signal Groups



Command Interface

- Address, bank, CS#, CKE, ODT, RAS#, CAS#, WE#
- MC → PHY; alert feedback PHY → MC

Write Data Interface

- `dfi_wrdata`, `dfi_wrdata_en`, `dfi_wrdata_mask`
- CRC and ECC sideband signals

Read Data Interface

- `dfi_rddata_en` MC → PHY (look-ahead)
- `dfi_rddata`, `dfi_rddata_valid` PHY → MC

Update / Status / PHY Managed

- Handshake: `ctrlupd`, `phyupd`, `phymngd`
- Freq ratio, init start/complete, error reporting

DFI: The Standard MC-PHY Interface

- **DFI** is the industry-standard interface between MC and DDR PHY
- Decouples MC from PHY implementation details
- Covers command, data, training and power signals
- DFI 5.1: PHY-independent boot, expanded freq range, enhanced calibration feedback



Ver	Key Addition	Support
1.0	Initial spec	DDR, DDR2
2.0	Training interface	DDR3, LPDDR2
3.0	Low-power handshake	DDR4, LPDDR4
4.0	DDR4/LPDDR4 paths	DDR4, LPDDR4
5.0	PHY-independent boot	DDR5, LPDDR5
5.1	Full back-compat	All gens

DFI 5.1 Key Features

Three Pillars of DFI 5.1

- ❶ **PHY-Independent Boot**
MC initialises DRAM without knowing PHY internals
- ❷ **Expanded Frequency Range**
Covers low-power LPDDR through DDR5-6400
- ❸ **Enhanced PHY↔MC Interaction**
Real-time calibration feedback and margining

Backward Compatibility

- Supports DDR through DDR5 and LPDDR through LPDDR5
- One DFI 5.1 MC paired with generation-specific PHY covers every standard

Why it matters for our MC

- Single RTL targets DDR3, DDR4, DDR5 and LPDDR4
- Swapping PHY = different memory standard, same controller